Deploy Next.js App on AWS EC2 With SSL & Custom Domain

Commands from the supplied screenshots, with a brief explanation of each use case.

Workflow
Connect to EC2, install Node.js, clone and build Next.js, keep it running with PM2, proxy it through Nginx, and install SSL with Certbot.

Included sections

1. Connect to AWS EC2

Protect the key pair

```
chmod 400 your-key.pem
```

Allows only the local owner to read the private key, which is required by SSH security checks.

Connect to the instance

```
ssh -i your-key.pem ubuntu@your-ec2-public-ip
```

Opens an SSH session to the Ubuntu EC2 instance using the selected key pair.

2. Install Node.js and Git

Add the Node.js 20 package source

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

Downloads and runs the NodeSource setup script so Ubuntu can install Node.js 20 packages.

Install Node.js and Git

```
sudo apt-get install -y nodejs git
```

Installs Node.js, npm, and Git. The -y option automatically accepts the installation prompt.

3. Clone and build the Next.js project

Clone the repository

```
git clone https://github.com/yourusername/your-nextjs-project.git
```

Downloads the project source code from GitHub to the EC2 instance.

Enter the project directory

```
cd your-nextjs-project
```

Changes the current working directory to the cloned Next.js project.

Install project dependencies

```
npm install
```

Reads package.json and installs the packages required by the application.

Create or edit environment variables

```
nano .env
```

Opens the environment file so required API keys, database URLs, and configuration values can be entered.

Build the application

```
npm run build
```

Runs the project's production build script and creates the optimized Next.js build output.

4. Run Next.js with PM2

Install PM2 globally

```
sudo npm install pm2 -g
```

Installs PM2 as a global command so it can manage the Next.js process.

Start the application

```
pm2 start "npm start -- -H 0.0.0.0" --name Ai-roater
```

Runs npm start through PM2, binds the server to all network interfaces, and assigns the process name Ai-roater.

Save the PM2 process list

```
pm2 save
```

Stores the current PM2 process list for restoration after a server restart.

Configure PM2 startup

```
pm2 startup
```

Generates the startup instructions required to launch PM2 automatically when EC2 reboots.

5. Install and configure Nginx

Install Nginx

```
sudo apt install nginx -y
```

Installs Nginx so it can receive public HTTP requests and forward them to Next.js.

Open the default Nginx site

```
sudo nano /etc/nginx/sites-available/default
```

Opens Ubuntu's default Nginx server configuration for editing.

Clear the default file

```
sudo truncate -s 0 /etc/nginx/sites-available/default
```

Sets the default configuration file size to zero so a new server block can be entered.

Nginx configuration shown in the screenshots

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

The server listens on HTTP port 80 and forwards incoming requests to the Next.js server on port 3000. The proxy headers support host forwarding and upgraded connections.

Reload Nginx

```
sudo systemctl reload nginx
```

Applies the saved Nginx configuration without performing a full service stop and start.

6. Install SSL and add the custom domain

Refresh Ubuntu packages

```
sudo apt update
```

Downloads current package information before installing Snap support.

Install Snap

```
sudo apt install snapd -y
```

Installs the Snap package manager used by the screenshot workflow to install Certbot.

Install Snap core

```
sudo snap install core
```

Installs the base Snap runtime required by Certbot.

Refresh Snap core

```
sudo snap refresh core
```

Updates the core Snap runtime to its current available version.

Install Certbot

```
sudo snap install --classic certbot
```

Installs the Let's Encrypt Certbot client with classic system access.

6. Install SSL and add the custom domain - continued

Create the Certbot command link

```
sudo ln -s /snap/bin/certbot /usr/bin/certbot
```

Creates a command path so certbot can be run directly from the terminal.

Reopen the Nginx configuration

```
sudo nano /etc/nginx/sites-available/default
```

Opens the server block so server_name can be changed from the catch-all value to the custom domain.

In the Nginx configuration, replace the server_name value with the intended domain before reloading.

Reload Nginx after adding the domain

```
sudo systemctl reload nginx
```

Applies the updated server_name configuration.

Request and install the SSL certificate

```
sudo certbot --nginx -d app.localhostak.online
```

Requests a Let's Encrypt certificate for the domain and asks Certbot to update the matching Nginx server block.

Completed setup
The screenshot workflow is complete after the Next.js process is running under PM2, Nginx forwards domain traffic to port 3000, and Certbot installs the domain certificate.